

# PORTFOLIO



## 정승환

안정적인 서비스 운영을 위해, 근거 있는 시스템 설계를 추구하는  
백엔드 엔지니어 정승환입니다.



## 정승환 SeungHwan Jung

안녕하세요. 안정적인 서비스 운영을 위해, 근거 있는 시스템 설계를 추구하는  
백엔드 엔지니어 정승환 입니다.

### 😊 개인정보

생년월일 | 1998년 11월 15일

전화번호 | 010-2942-1408

이메일 | ekrmhdsl7@gmail.com

Github | <https://github.com/WAFriend3416>

## 프로젝트

### 1. 도장콕

- 임차인의 부동산 임대 계약을 AI로 지원하는 모바일 서비스 (계약서 분석 · AI 체크리스트 · 매물 커뮤니티)
- 카카오테크 부트캠프 파이널 프로젝트 | 클라우드 인프라 설계, K8S 클러스터 구축·운영, CI/CD 자동화

### 2. DC2(Developer Community Club)

- 게시글 · 이미지 업로드 기능을 갖춘 개발자 커뮤니티 웹앱
- 개인 프로젝트 | 시스템 설계, AI 활용 백엔드 개발, 부하 테스트, 무중단 배포 자동화

### 3. 고립 어르신 고독사 탐지 및 예방 시스템 고도화

- 1인 가구 어르신을 IoT 센서·관제·AI 리포팅으로 모니터링하는 고독사 예방·탐지 시스템
- 한전MCS 산학협력 프로젝트 | PM, 시스템 설계, 클라우드 인프라 관리

## 오픈소스 기여

- OpenStack CLI - ``server evacuate --wait`` 명령의 SHUTOFF 인스턴스 무한 대기 버그 수정
  - 공식 머지 (v8.1.0 2025.05) ([링크](#))
- Cilium - `cilium-dbg bpf ipcache get` 명령의 IP prefix 매칭 로직을 net/netip 기반으로 전환
  - Upstream 공식 머지 (#46181, 2026.06) ([링크](#))

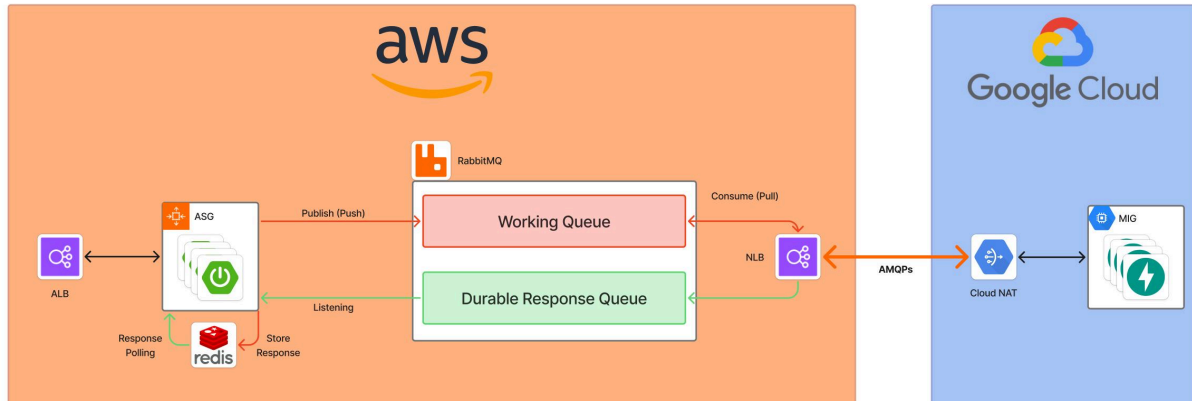
## 도장콧 자체 LLM 운영을 위한 멀티클라우드 아키텍처

해결 과제 | 자체 LLM 운영에 필요한 **GPU 비용 부담**과 긴 응답 시간으로 인한 **동기 통신 병목**

해결 방안 | **AWS-GCP 멀티클라우드 도입** 및 **메시지 큐 기반 비동기 AI 처리 구조** 구축

적용 결과 | GPU 비용 부담 분산, AI 서버 외부 직접 접근 차단, WAS 스레드 장시간 점유 완화

### 구성 | 메시지 큐 기반 AI 기능 처리 흐름



AMQPS 통신을 통한 접근 통제 및 비동기 처리 아키텍처

### 배경 | 프로젝트 제약사항

- 외부 AI API 사용 시 개인정보 노출 위험으로 자체 호스팅 LLM 필요
- AWS GPU 온디맨드 비용 부담으로 **GCP 크레딧 기반 GPU 서버 분리 결정**
- 초기 부하 테스트에서 장기 AI/OCR 동기 처리 한계 확인: 87.5% 502, 평균 39.7초 지연

### 설계 | 주요 고려사항

#### 1. 멀티클라우드 간 통신 방식 선택

- LLM 추론이 **30초~1분 이상** 소요되어 동기 HTTP 대신 비동기 처리 방식 비교
- 운영 복잡도는 증가하지만, 응답 매칭과 메시지 연속성이 안정성에 더 중요하다고 판단하여 RabbitMQ 채택

| 대안              | 트레이드 오프                                                     |
|-----------------|-------------------------------------------------------------|
| 동기 HTTP         | 구현은 단순하지만 <b>504 타임아웃</b> 과 WAS 스레드 장시간 점유 발생               |
| HTTP Callback   | 비동기 응답은 가능하지만 콜백 라우팅 복잡도와 추가 포트 오픈 부담                       |
| Redis Pub/Sub   | Redis 인프라 재활용 가능하지만 메시지 연속성 없음                              |
| RabbitMQ 비동기 통신 | 메시지 ID 기반 응답 매칭과 <b>Durable Queue</b> 로 안정성 확보, MQ 운영 부담 추가 |

#### 2. 멀티클라우드 통신 보안 강화

- AMQP 특성상 **ALB(L7)** 대신 **NLB(L4)** 기반 AMQPS 경로 선택
- GCP Cloud NAT Static IP**를 AWS SG에 등록하고, RabbitMQ 계정 인증으로 접근 통제

### 결과 | 구조적 분리를 통한 비용·보안·안정성 개선

- [보안] GCP AI 서버 **Public IP 제거** 및 **Inbound All Deny**로 외부 직접 접근 차단
- [비용] GCP GPU 서버 분리로 **AWS GPU 온디맨드 비용 부담 완화**
- [안정성] 비동기 처리로 **WAS 스레드 장시간 점유**와 **API 연쇄 장애 위험** 완화

## 도장국 | 보안그룹 접근 제어를 K8S 트래픽 흐름에 맞게 재정의

설계 과제 | EC2 보안그룹식 접근 제어를 K8S 서비스 경로에 맞게 재구성

설계 방향 | ALB→NGF→ClusterIP 진입 경로를 고정하고, SG와 NetworkPolicy 책임 분리

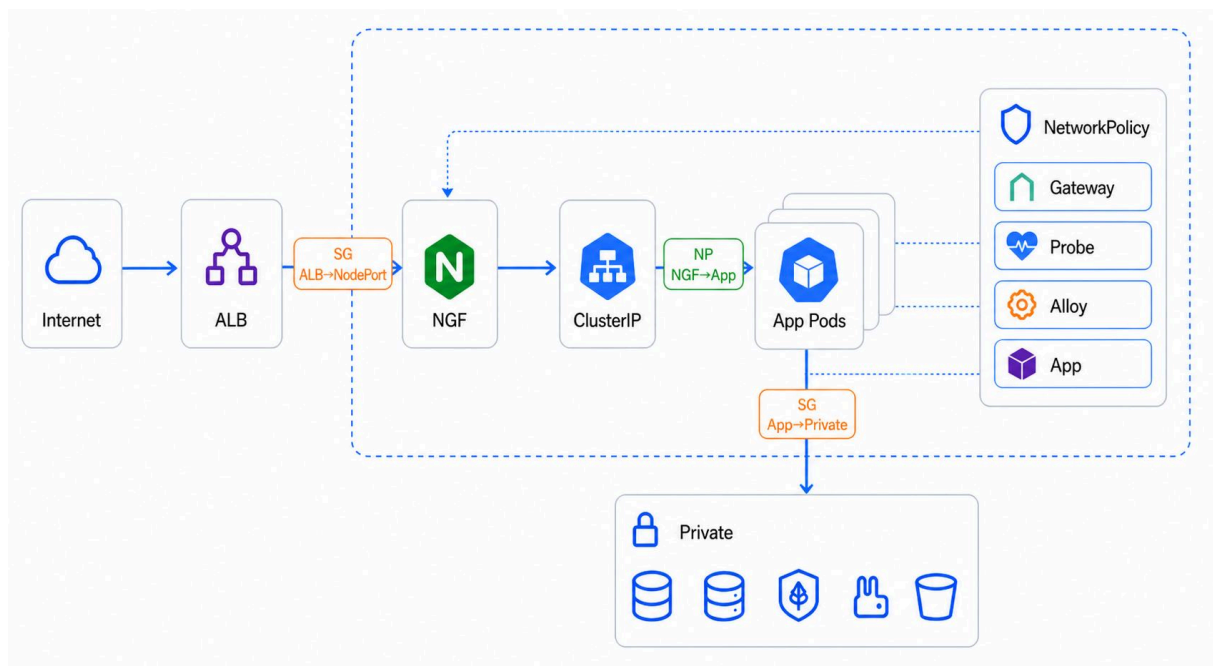
### 선택 | ALB→NGF→ClusterIP 진입 경로 구성

- 외부 요청은 Pod에 직접 닿지 않고 **Gateway**를 거쳐 앱 **Service**로 전달되도록 구성
- LoadBalancer 자동 프로비저닝 대신 **ALB NodePort + NGINX Gateway Fabric** 조합 선택
- NGF는 Deployment 대신 **DaemonSet**으로 배치해 노드별 로컬 진입점 유지
- ALB는 TLS 종료·health check, NGF는 HTTPRoute 기반 FE/BE/AI 라우팅 담당

### 조정 | NetworkPolicy 적용 범위 재정의

- SG 에서 적용한 **default deny**를 그대로 적용하면 K8S Service 경로와 Pod selector 경계가 맞지 않음
- Calico egress는 **Service VIP가 Pod IP**로 변환되기 전 단계에서 평가되어 selector 기반 허용과 충돌
- **egress deny**는 제외하고 ingress 중심으로 Gateway, kubelet probe, Alloy 수집 경로 허용
- 외부 진입, Pod 간 통신, 운영 수집 경로를 분리해 필요한 ingress만 문서화

### 경로 | 외부 요청과 Pod ingress 경계



외부 진입 경로와 네트워크 통제 지점

- ALB는 외부 진입과 TLS 종료, NGF는 HTTPRoute 라우팅 담당
- NGF는 Worker 전체에 배치해 단일 Gateway Pod 장애 영향 축소
- 라우팅 규칙은 HTTPRoute로 GitOps 관리

### 결과 | SG와 NetworkPolicy 책임 분리

- **SG**는 ALB→NodePort와 App→Private 리소스 경계 담당
- **NetworkPolicy**는 Gateway, probe, Alloy 등 Pod ingress 경계 담당
- 외부 요청은 **ALB→NGF→ClusterIP** 경로로만 앱 Pod에 전달

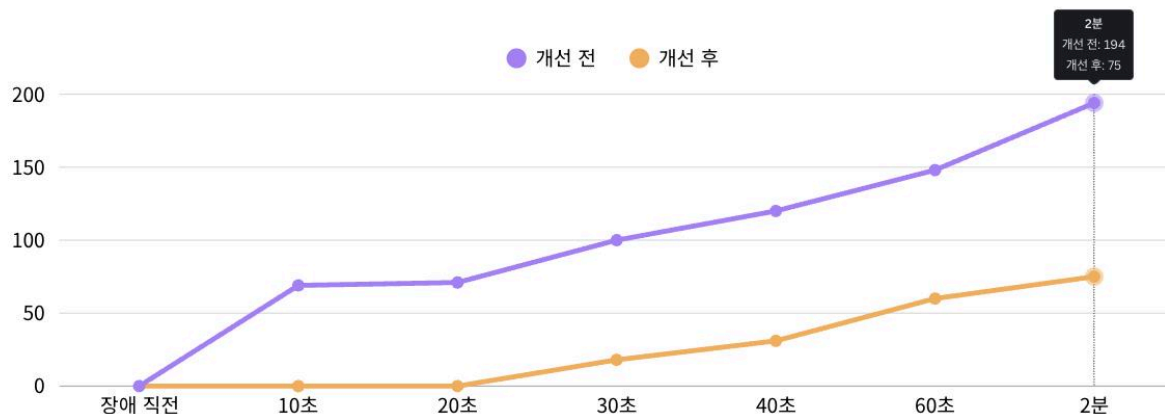
## 도장록 Fault Injection으로 검증한 K8S 클러스터 AZ 장애 내성

검증 과제 | AZ 장애 상황에서 주요 워크로드 유지 여부 확인

검증 방식 | AWS FIS로 AZ 장애를 주입하고, 부하 상황에서 복구 흐름 관찰

검증 결과 | 장애 노드 탐지 간격을 줄여 초기 실패 구간 제거 / ALB 헬스체크 단축은 503 급증으로 미적용

### 장애 주입 직후 2분간 구간별 실패 건수



AZ 장애는 10분간 주입, 그래프는 장애 직후 라우팅 지연이 집중된 초기 2분 구간만 표시.

### 결과 | Node NotReady 감지 단축으로 초기 실패 구간 개선

- 장애 노드 감지 시간을 40초→20초 수준으로 단축
- 노드 상태 갱신 주기를 10초→4초로 조정
- 초기 20초 실패 71건→0건, 502 69→7건으로 90% 감소
- 전체 실패 194→75건으로 61% 감소
- ALB 헬스체크 단축은 503 0→293건으로 역효과 확인

### 검증 | FIS로 AZ 장애 상황 재현

- CP 1대 + Worker 6대, 3-AZ × 2 구성에서 AZ 2c Worker Node 2대 10분 중지
- k6 30VU 부하 유지, 장애 직후 초기 2분의 누적 실패 건수 비교

### 원인 | K8S와 ALB의 장애 감지 기준 차이

- 502는 NGF가 아직 제거되지 않은 Pod endpoint로 연결하며 발생
- 504는 ALB가 NodePort health check 기준으로 장애 Node에 라우팅하며 발생

### 한계 | ALB NodePort 경로의 Pod 단위 라우팅 제약

- 외부 트래픽은 ALB → Worker Node → NGF Pod → Backend Pod 경로로 유입
- ALB는 Pod readiness가 아니라 Worker Node의 NodePort health check 기준으로 target 판단
- Pod 단위 라우팅 구조는 대안이지만, self-managed kubeadm 범위에서는 미적용

## 도장국 모니터링 장애에도 끊기지 않는 관측성 구조

해결 과제 | 모니터링 서버 장애 시 메트릭·로그·대시보드 접근까지 함께 끊기는 **SPOF** 제거

해결 방안 | Alloy Dual-Write와 ALB·Thanos 기반 조회 경로 이중화

검증 결과 | 모니터링 EC2 1대 중지 중 **Grafana** 접속 우회, 메트릭·로그 수집 유지, 관측 경로 자동 복귀

### 상황 | 관측 시스템 자체의 단일 장애점

- 기존 모니터링 스택이 단일 EC2에 집중되어 관측 시스템 자체가 **SPOF**로 남음
- 장애 분석에 필요한 메트릭·로그·대시보드 접근 경로도 함께 중단될 수 있음

### 조치 | 관측 데이터와 조회 경로 이중화

- 모니터링 EC2 2대에 Prometheus·Loki·Grafana 스택 분리 배치
- Alloy가 양쪽 서버로 메트릭·로그 동시 전송
- Thanos Query는 `replica` 라벨 기준 중복 샘플 제거
- ALB가 Grafana 접속을 **healthy** 서버로 라우팅

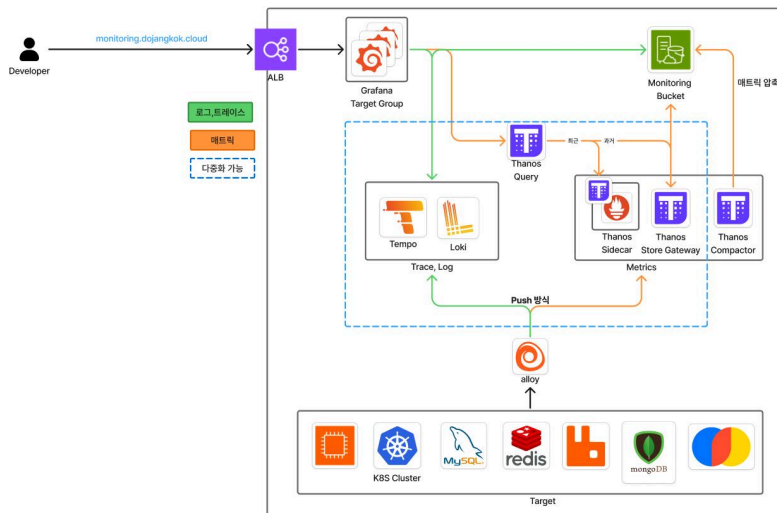
### 검증 | 장애 범위와 복구 시간 분리

| 대상         | 복구 시간  | 관측 영향                                                 |
|------------|--------|-------------------------------------------------------|
| Alloy      | 약 3초   | 해당 서버 수집만 중단, <b>Dual-Write</b> 로 <b>monitor-a</b> 정상 |
| Grafana    | 18초    | Prometheus 독립 수집으로 <b>메트릭 유실 0건</b> , 대시보드 일시 중단      |
| Prometheus | 약 97초  | <b>scrape</b> 누락 발생, Thanos S3 풀백으로 과거 데이터 조회 정상      |
| Loki       | 약 119초 | Alloy 버퍼링·재전송으로 장애 구간 <b>BE</b> 로그 유실 0건              |

### 결과 | 한쪽 모니터링 서버 장애에도 관측 유지

- 장애 중: ALB가 정상 모니터링 서버로 Grafana 접속 우회
- 수집 유지: Alloy Dual-Write로 메트릭·로그 수집 지속, 대시보드 **30분 범위에서 메트릭 누락 구간 없음**
- 복구 확인: HealthyHostCount **2→1→2**, 중지 서버 재기동 후 관측 경로 자동 복귀

### 구성 | 수집·조회·보관 경로 분리



모니터링 시스템 아키텍처 다이어그램

## 도장국 Public 레포 기반 GitOps와 Secret 외부화

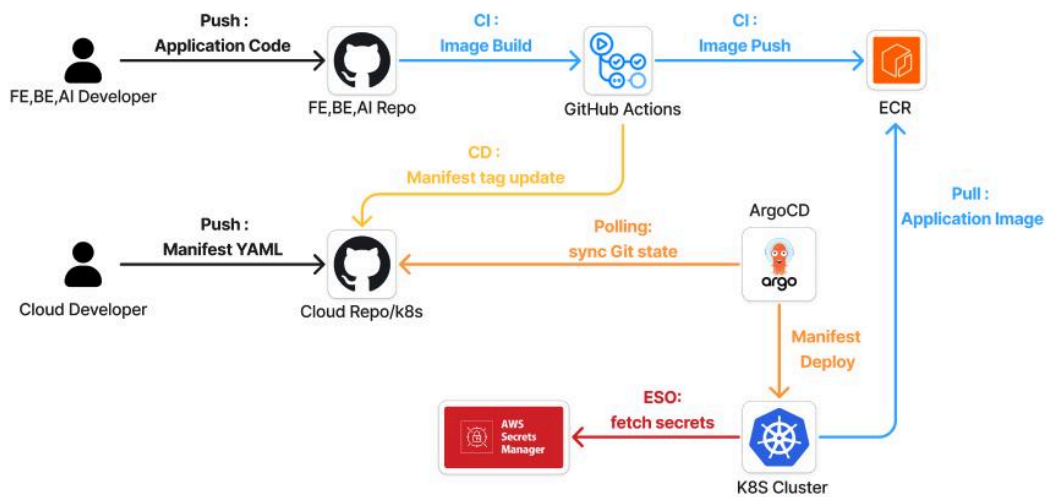
해결 과제 | Public 레포 기반 GitOps에서 **Secret 값을 Git에 두지 않고** K8S 배포 자동화

설계 요약 | **App Repo·Manifest Repo·ArgoCD·ESO**로 배포 상태와 Secret 값의 책임 분리

### 문제 상황 | GitOps 단일 원천과 Secret 비노출의 충돌

- GitOps: Git을 배포 상태의 단일 원천으로 사용
- Public 레포 제약: Secret 값 또는 암호문 커밋 불가
- 핵심 충돌: GitOps는 Git에 모든 배포 상태를 두려 하고, Secret 보안은 값을 Git 밖에 두도록 요구

### 구성 | 배포 상태와 Secret 값의 책임 분리



Public 레포 기반 GitOps와 Secret 외부화 흐름

### 설계 | Public 레포 제약 아래 Secret 외부화 방식 비교

- 기준 1: Git에 평문 Secret은 물론 암호문도 남기지 않을 것
- 기준 2: GitOps의 단일 원천 구조를 깨지 않을 것
- 기준 3: Secret 값 변경이 클러스터에 자동 반영될 것

| 방식                    | 트레이드 오프                                                            |
|-----------------------|--------------------------------------------------------------------|
| SOPS / Sealed Secrets | 암호문이 Git에 남아 Public 레포 제약과 충돌                                      |
| CI 직접 주입              | Secret 값은 숨길 수 있지만, 배포 상태가 Git 밖 CI 실행에 의존                         |
| ESO                   | Git에는 <b>ExternalSecret</b> 참조만 남고, 실제 값은 AWS Secrets Manager에서 관리 |

### 결과 | Git에는 Secret 참조만, 실제 값은 외부 Secret Store에 보관

- Git 커밋 대상: Deployment / Service / ExternalSecret 정의
- Git 제외 대상: DB·Redis·JWT 등 실제 Secret 값
- 동기화 책임: **ArgoCD**는 배포 상태, **ESO**는 **Secret** 값 반영

## 도장국 HPA와 ArgoCD Auto-Sync의 반복 스케일링 충돌 해소

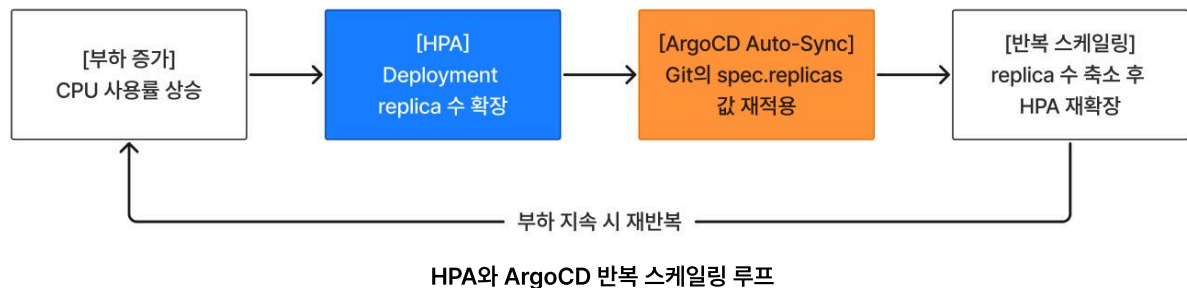
**해결 과제** | HPA와 ArgoCD Auto-Sync가 spec.replicas를 동시에 관리해 부하 중 반복 스케일링 발생

**해결 방안** | spec.replicas 필드를 매니페스트에서 제거해 HPA가 단독으로 관리하도록 위임

**적용 결과** | 동일 부하 비교에서 반복 스케일링 95% 감소, GitOps로 인한 replica 변경 제거

### 문제 구조 | GitOps와 HPA의 동일 필드 충돌

- HPA: 부하 증가 시 대상 Deployment의 replica 수 확장
- ArgoCD Auto-Sync: Git 매니페스트의 `spec.replicas` 값으로 replica 수 복원
- 반복 조건: 부하 지속 + Git 기준 replica 재적용
- 원인: `spec.replicas` 관리 책임이 HPA와 GitOps에 동시에 존재

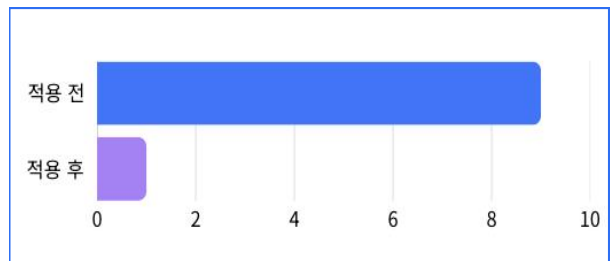
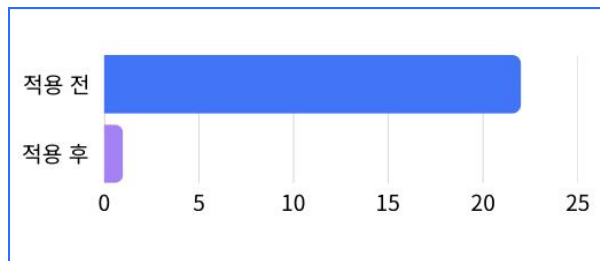


### 조치 | replica 관리 책임 분리

- Deployment 매니페스트에서 `spec.replicas` 제거
- replica 수 결정은 HPA에 위임
- GitOps는 이미지·환경변수·probe 등 배포 형상 관리에 집중

### 결과 | 반복 스케일링 감소와 replica 변경 제거

- 측정 조건: 5분 부하 인가 중 30초 간격 sync 루프 실행, 변경 전후 동일 조건 비교
- `events.log`: ScalingReplicaSet 이벤트 22회 → 1회 (95% 감소)
- `sync.log`: GitOps replica 변경 9회 → 1회



## DC2 백만 MAU 기준 AWS 3-Tier 아키텍처 설계·검증

해결 과제 | 백만 MAU 트래픽을 안정적으로 처리하기 위한 확장 가능하고 운영 안정적인 인프라 구조

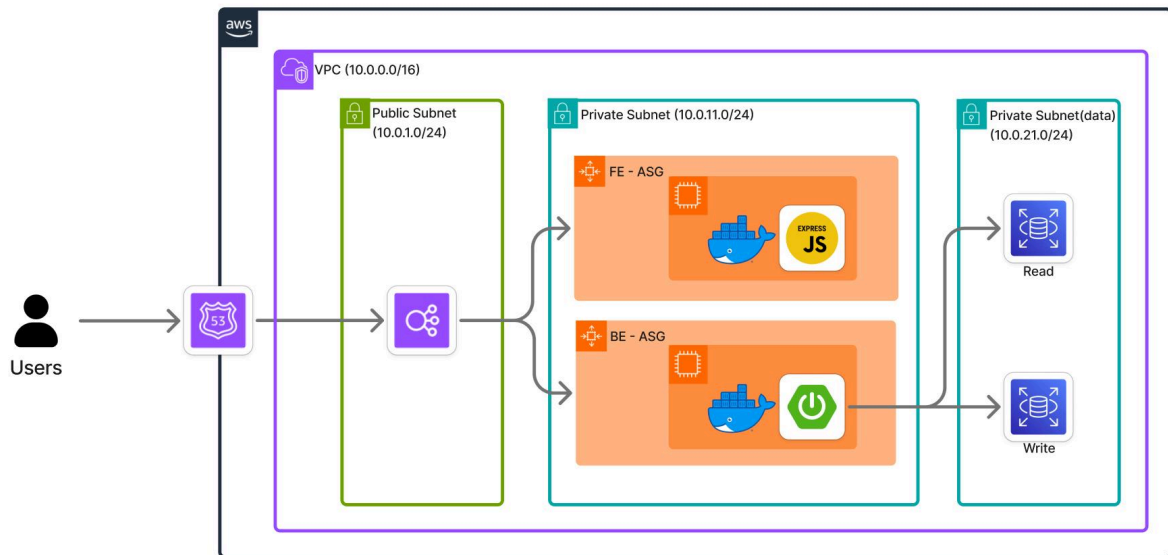
해결 방안 | Public ALB + Private App/Data 계층으로 분리한 AWS 3-Tier 구조 적용

검증 결과 | ASG 4대 기준 466.6 RPS로 목표 대비 약 1.5배 처리 여유 확보

### 배경 | 검증 목표

- 개발자 커뮤니티 웹앱 기준 목표 처리량을 약 **300 RPS** 로 산정
- 목표 처리량을 기준으로 3-Tier 인프라를 설계하고 부하 테스트로 처리 가능성 검증

### 구현 아키텍처 | AWS 3-Tier



Public ALB와 Private App/Data 계층으로 분리한 요청 처리 구조

### 설계 근거 | 과제별 대응

- 보안 : 외부 접근 지점은 **ALB**로 제한, App/Data 계층은 **Private Subnet**에 배치
- 확장성 : **ASG** 수평 확장과 **RDS Primary/Read Replica** 분리로 읽기 부하 분산
- 운영 : **VPC Endpoint**로 Private Subnet 내부의 배포·운영 경로 확보

### 검증 결과 | 부하 테스트

- 단일 BE 인스턴스 **327 RPS** 처리 → 목표 평균 약 **300 RPS** 충족
- **ASG 1→4대** 확장 구성에서 **466.6 RPS** · 전체 **p95 110.92ms** · 오류율 **0.07%** 확인
- 통합 처리량 기준 목표 대비 약 **1.5배** 여유 확보
- 단일 BE 병목 원인은 **BCrypt** 해싱 연산으로 확인 (로그인 API p95 413ms)

※ 검증 범위 : 단일 AZ 기준 성능 검증 / Multi-AZ 고가용성 구성은 프로젝트 범위에서 제외

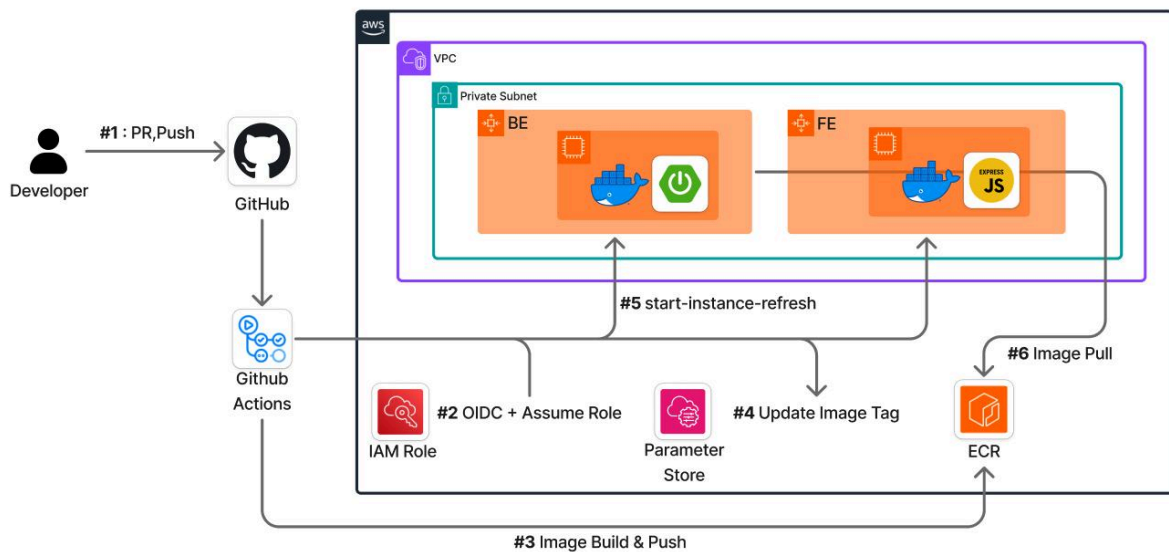
## DC2 ASG 기반 무중단 배포 자동화

해결 과제 | Private Subnet FE/BE 인스턴스를 외부 SSH 없이 교체하면서 사용자 경험 영향 최소화

해결 방안 | GitHub Actions OIDC + ASG Instance Refresh로 무중단 CI/CD 파이프라인 구축

검증 결과 | ASG Instance Refresh 동안 읽기 API 100% 성공, 처리량 드롭 0건

### 구성 | CI/CD 파이프라인 흐름



OIDC와 ASG Instance Refresh로 구성된 외부 접속 없는 무중단 배포 흐름

### 검증 목표

- 코드 배포 중 사용자 경험 영향 최소화
- Private Subnet 인스턴스를 외부 노출 없이 교체할 수 있는 경로 필요

### 검증 기준 및 측정 지표

- Instance Refresh 성공 여부 및 소요 시간
- 읽기 API 성공률, p95 응답 시간, 처리량 드롭 여부

### 무중단 배포 검증 결과

- 조건: 20분 부하 유지 중 ASG Instance Refresh 실행
- 정책: MinHealthy 50%, Warmup 180s
- 결과: Instance Refresh 약 4분 50초, 100% Successful
- 읽기 API 100% 성공 (목록 p95 32ms / 상세 p95 41ms)
- 처리량 드롭 0건

## 프로젝트 배경

- 한전MCS의 전력 기반 독거노인 안부 확인 시스템을 고도화하는 산학협력 과제
- 기존 전력 사용 데이터와 전등 점등 여부만으로는 **어르신의 생활 안전·위기 상황을 판단하는 데 한계**
- IoT 팀은 스마트 전등 데이터에 **재실·소음감지 센서 데이터**를 보강하는 역할을 담당

## 현실적 제약과 전환

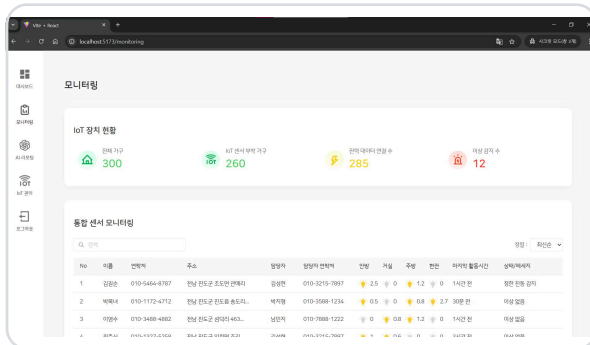
- 초기 안은 바닥 진동 기반 활동·이상 탐지였으나, 생활 진동 노이즈와 자체 낙상 실험 부담이 컸음
- 어르신 생활공간에 낯선 장치를 추가 설치하는 부담을 고려해, 기존 스마트 전등과 결합 가능한 센서 조합을 검토
- 점등 여부만으로 부족했던 생활 신호를 보완하기 위해 재실·소음감지 센서를 추가하는 방향으로 전환
- 100여 가구의 센서·생활 패턴 데이터를 직접 처리하기엔 운영·접근 통제 부담이 크다고 판단
- 확장성·보안성·운영 부담을 고려해 클라우드 기반 수집 구조로 구축 방향을 전환

## 구성

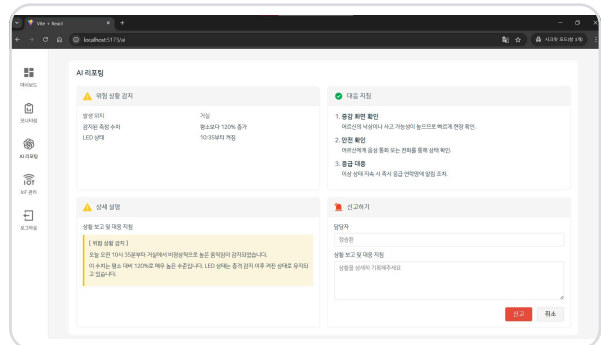
- **센서 데이터 수집** : 스마트 전등 ON/OFF와 재실·소음 상태를 LTE 통신으로 AWS IoT Core MQTT 토픽에 전송
- **버퍼링** : SQS로 통신 지연·메시지 버스트와 Lambda 처리량을 분리
- **정규화·저장** : Lambda가 센서 메시지를 파싱해 RDS(MariaDB)에 적재
- **조회·리포팅** : 대시보드와 AI 리포팅 화면이 동일한 센서 데이터를 참조

## 구현 결과 | 모니터링 화면과 데이터 흐름

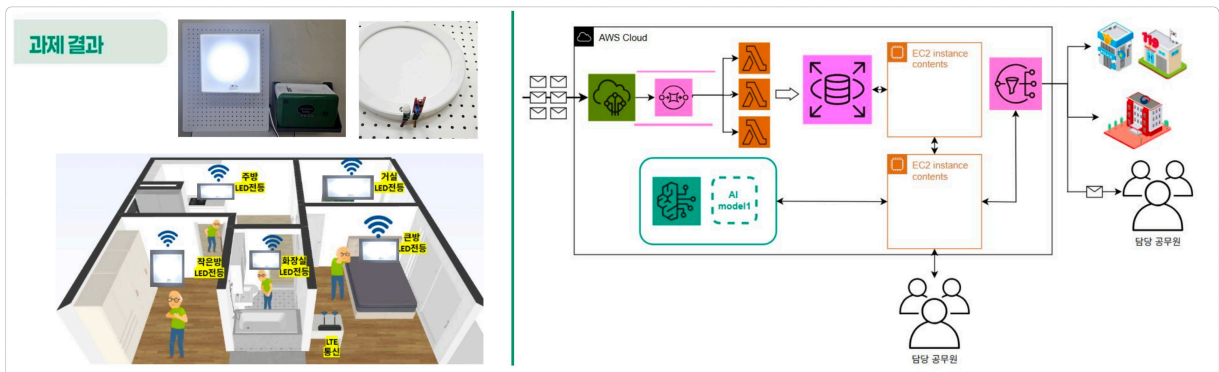
- 프로토타입 환경에서 센서 데이터가 수집·저장·조회되는 흐름을 확인
- 실서비스 배포까지는 이어지지 못했으나, 모니터링·AI 리포팅 화면에서 데이터 활용을 확인



가구·센서 상태 통합 모니터링



AI 리포팅 생성 페이지



추가 센서 데이터를 수집·저장·조회하기 위한 AWS 기반 파이프라인

## 부록 - 도장국 전체 아키텍처 다이어그램

